Rainier Ordinario - 301631145
Nelson Dang - 301578533
Derek Kang - 301618005
Kenneth Pamintuan - 301582028
CMPT 276 Group 2

ESCAPE GAME: Phase 4 Report

1. THE GAME

Overview

Escape is a tile-based Java game where players navigate four prison sections (Cell Block, Cafeteria, General Time, Outdoor Recreation) collecting coins, avoiding cops and traps, and gathering three shovel parts to escape. The core mechanic requires collecting a specific number of coins per level to unlock exits within a 300-tick time limit.

Features:
4 progressively complex levels, coin collection system, cop AI with patrol/chase mechanics, trap damage system, shovel part collection, real-time timer, health/score tracking, smooth camera, visual feedback (animations, vignettes, toast messages).

Plan vs Reality

Successfully implemented: all four prison sections, coin-based exit mechanics, cop pathfinding with diagonal movement, trap damage, shovel part requirements, 300-tick time pressure, win/lose conditions.

Deviations: removed corruptible cop bribe mechanic (simplified gameplay), adjusted coin requirements based on map reachability, added visual polish not in original plan.

What Changed and Why

Coin System: The original plan didn't account for unreachable map tiles in sealed rooms. We discovered Level 4 had 4 unreachable 'I' tiles in sealed rooms using BFS pathfinding validation. We corrected this by: validating reachable coins for each level, adjusting coin requirements, and adding programmatic coin spawns as a workaround.

Cop Spawning: Cop Spawning Validation: Initial spawns placed cops at hard-coded positions like (15,8) and (28,10) which were walls on Level 4. We implemented findNearestWalkable() using BFS to automatically move cops to the nearest walkable cell. Testing confirmed 100% cop placement success rate with zero wall collisions.

Visual Polish: Added animations, vignette effects for damage/healing, and toast messages for improved feedback.

Key Lessons

1. Map Design is Critical - unreachable tiles break mechanics; always validate pathfinding before finalizing levels.
2. Separation of Concerns - Factory Method pattern kept game engine clean and levels easy to test independently.

3. Iterative Testing - early playtesting on Level 3 revealed all 4 'I' tiles were unreachable, forcing us to add programmatic spawns. Without testing before deadline, these levels would have been unwinnable. We caught this in Phase 3 and fixed before Phase 4.
4. Team Communication - clear division of labor enabled parallel development but required careful API alignment.
5. Technical Debt Payoff - early refactoring (Coordinate class, Board abstraction) significantly improved maintainability.

2. TUTORIAL & HOW TO PLAY

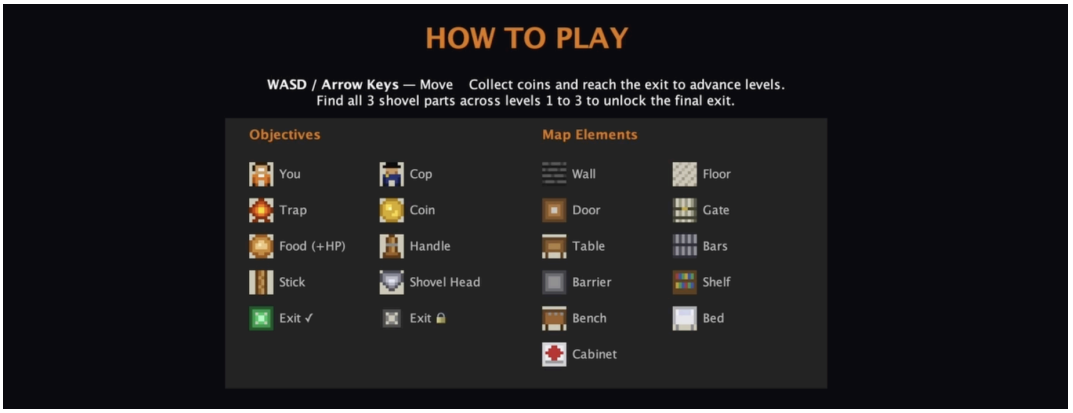
Starting: Launch the game to appear in Level 1 (Cell Block). HUD shows coins required (top-right), health (top-left, starts at 100), timer (remaining ticks), shovel parts collected, and in-game legend (tile types).

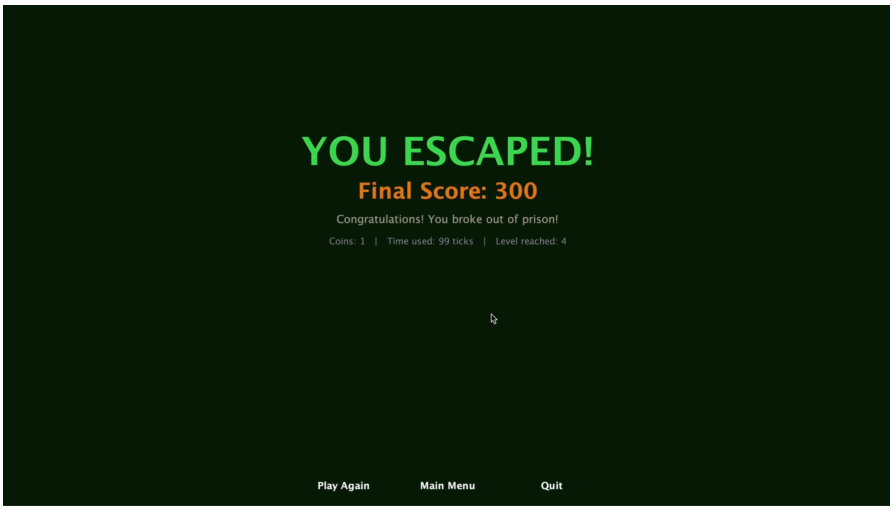
Controls: W/↑ up, A/← left, S/↓ down, D/→ right

Mechanics:

- Coins - Collect required amounts to unlock exits: L1: 2, L2: 3, L3: 3, L4: 4. Exit locks without enough coins.
- Cops - Deal 10 damage on contact. Patrol in patterns; plan routes to avoid them.
- Traps - Deal 10 damage when stepped on. Hazards, not walls; can walk through but take damage.
- Shovel Parts - Collect all three by Level 4 (Handle in L1, Stick in L2, Shovel Head in L3) to unlock final exit.
- Health & Time - Health starts at 100; reaching 0 = loss. 300 ticks per level; timeout = loss. Food restores 20 HP. Score increases with coins.

Win: Collect required coins + find shovel part → reach exit (L1-3). Collect required coins with all 3 parts → reach exit (L4).
Lose: Health drops to 0 or timer reaches 300 ticks.





3. BUILD AUTOMATION

Prerequisites: Java 17+, Maven 3.8+

To build and run the game:
mvn clean test package javadoc:javadoc
java -jar target/escape-game-1.0-SNAPSHOT.jar

SUMMARY
Escape Game is a complete, playable tile-based puzzle game demonstrating solid software engineering: Factory Method pattern, comprehensive testing, clean separation of concerns, and automated artifact generation. Fully functional and ready to play. Anyone can clone the repo and run mvn clean test package javadoc:javadoc to generate the executable JAR and documentation.